

Indian Institute of Technology Jodhpur

Department of Computer Science and Engineering

Assignment 2

Cost-Efficient Segmentation of the Main Object with a Tiny Fully Convolutional Network

Abhyudaya Tiwari

B23CS1085

Supervisor: Deepak Mishra

November 1, 2026

Contents

1 Understanding of the Task **1**

2 Data and Experimental Setup **2**

2.1 Datasets and Split 2

2.2 Hardware and Software 2

2.3 Rules Followed 2

3 Network Architecture **3**

3.1 Depthwise Separable Block 3

3.2 Global Context 4

3.3 Layer by Layer 5

4 Training **6**

4.1 Loss 6

4.2 Optimisation and Augmentation 6

5 Evaluation Metrics **8**

5.1 Mean Intersection over Union 8

5.2 Boundary F1 8

6 Keeping the Pipeline Fast **9**

6.1 Speed Measures 9

6.2 Parameter File 9

6.3 Timing 10

7 Design Choices on the Validation Set **11**

8 Results **12**

8.1 Validation versus DUTS-TE 12

9 Captured Photos **13**

10 Failure Cases **14**

10.1 Camouflage in a Cluttered Outdoor Scene 14

10.2 Small Object on a Large Textured Surface 15
10.3 Object Touching the Border under a Colour Cast 16
10.4 Fine Structures and Boundary Accuracy 16

11 Conclusion **17**

Chapter 1

Understanding of the Task

The goal is a pipeline that labels every pixel of a photo as *main object* or *background* while using on average at most 50 ms of CPU time on a single core. The score to maximise is the Cost-Efficient Segmentation (CES) score, which rewards quality and charges a little for time and for the size of the parameter file:

$$Q = 0.70 \cdot \mathrm{mIoU} + 0.30 \cdot \mathrm{BF1}$$

$$\mathrm{CES} = Q \cdot \left[\, 1 - 0.20 \cdot T_{\mathrm{CPU}} / 50\ \mathrm{ms} - 0.10 \cdot \min(1, S_{\mathrm{disk}} / 4\ \mathrm{MiB}) \,\right] \quad \text{if } T_{\mathrm{CPU}} \le 50\ \mathrm{ms}, \text{ otherwise } 0.$$

I followed **Track B**: a small fully convolutional network (a U-Net) trained from scratch in PyTorch. The main stages are:

1. Resize DUTS photos and masks to 256 × 256 and split DUTS-TR into training and validation sets.
2. Design an encoder-decoder network with skip connections that is small enough for the CPU budget.
3. Train it on the GPU with pixel-wise cross entropy, soft Dice and a boundary weight.
4. Choose the architecture and loss on the validation set by an estimate of CES.
5. Make the CPU pipeline fast: fold BatchNorm, use channels-last TorchScript, store int8 weights.
6. Measure T_{CPU} on all 5,019 DUTS-TE photos, evaluate quality, and analyse failures.

Because time and file size cost only a small fraction of Q in this budget range (1 ms costs 0.4% of Q and 100 KB costs about 0.24%), the best trade-off is the most accurate network that still runs in a few milliseconds.

Chapter 2

Data and Experimental Setup

2.1 Datasets and Split

DUTS masks mark the main object or objects of each photo in white. Every photo is resized to 256 × 256 with `cv2.INTER_AREA` and every mask with `cv2.INTER_NEAREST`; a mask pixel is object if its value is at least 128. All metrics are computed on these 256 × 256 masks. DUTS-TR was split once with a fixed random seed (2024).

Table 2.1: Data splits.

Set	Source	Photos	Used for
Training	DUTS-TR	9,500	learning the weights
Validation	DUTS-TR (held out)	1,053	every design choice and the epoch
Test	DUTS-TE	5,019	final metrics and T_{CPU} only
Captured	own photos	4	testing only (never trained on)

The resized data is cached once as memory-mapped uint8 arrays (`src/prepare_data.py`). Photos stay in OpenCV's BGR order end to end, so the pipeline needs no colour conversion.

2.2 Hardware and Software

Table 2.2: Hardware and software.

Item	Value
CPU (timing)	AMD Ryzen 5 5600H with Radeon Graphics, 1 thread
GPU (training only)	NVIDIA GeForce GTX 1650, 4 GB
Software	Python 3.13, PyTorch 2.14.1+cu126, OpenCV 5.0, NumPy 2.5
Operating system	Windows 11

2.3 Rules Followed

- No pretrained weights and no library code for segmentation, clustering, morphology or metrics. The network, losses and metrics are written from scratch with PyTorch tensors.
- DUTS-TE is never used for training, tuning or choosing the epoch.
- The timed pipeline runs on the CPU with one thread: the four thread environment variables are set before NumPy, OpenCV and PyTorch are imported, followed by `cv2.setNumThreads(1)` and `torch.set_num_threads(1)`.
- All learned values are in the parameter file `weights/model.pt` (plain `torch.save`, no extra compression).

Chapter 3

Network Architecture

The network is a small U-Net. The **encoder** shrinks the 128 × 128 input four times by strided convolutions, the **decoder** brings the features back to 64 × 64 with bilinear upsampling, and **skip connections** concatenate encoder features of the same size into the decoder (as in U-Net). Figure 3.1 shows the whole network.

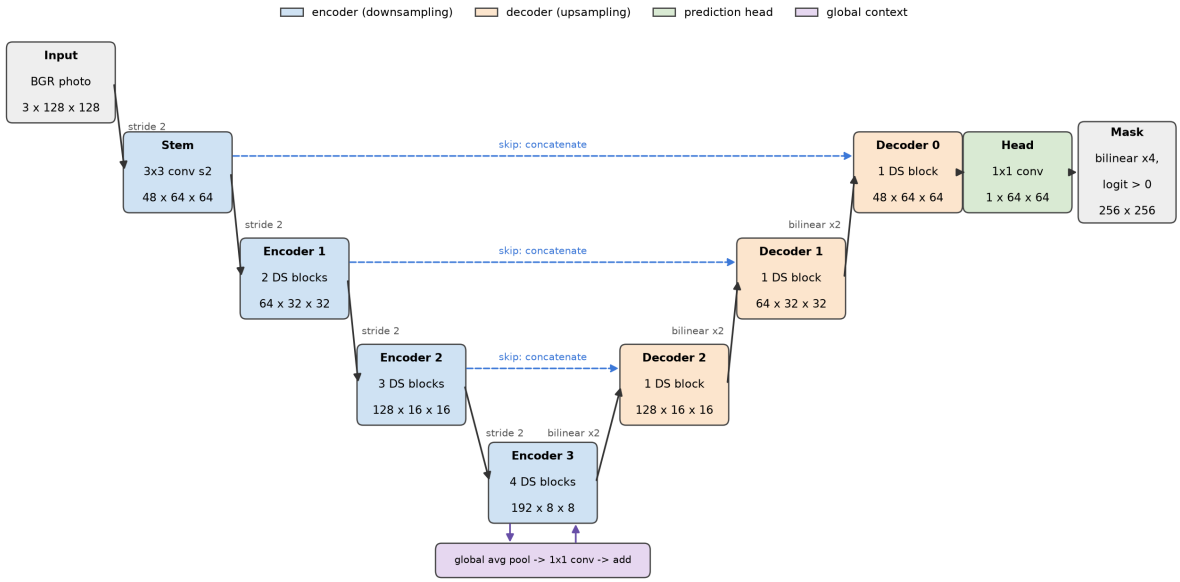


Figure 3.1: TinyUNet. Boxes give the output shape (channels × height × width) for a 128 × 128 input. Blue dashed arrows are skip connections.

3.1 Depthwise Separable Block

Every encoder and decoder stage is built from the depthwise separable (DS) block of Figure 3.2. A 3 × 3 *depthwise* convolution filters each channel on its own and a 1 × 1 *pointwise* convolution mixes channels. For $C_{in} = C_{out} = C$ this costs $9C + C^2$ weights instead of $9C^2$ for a normal 3 × 3 convolution, which is what makes the network fast enough. When the shape does not change, a residual connection adds the block input to its output.

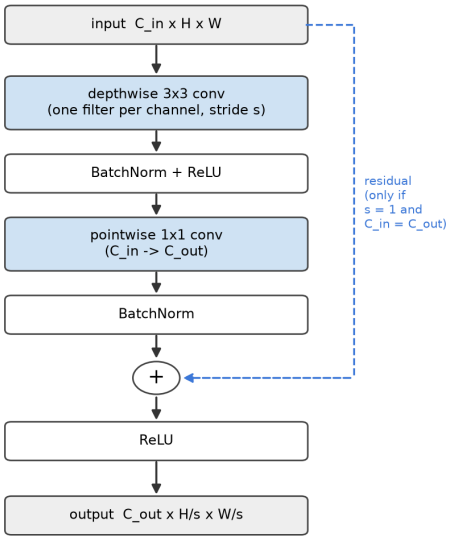


Figure 3.2: Depthwise separable block (stride s is 2 in the first block of each encoder stage, otherwise 1).

3.2 Global Context

Deciding which object is the *main* one needs image-level information. The deepest features (192 × 8 × 8) are averaged over the whole image, passed through a 1 × 1 convolution and added back to every position. This costs about 37K weights and almost no time.

3.3 Layer by Layer

Table 3.1: Layers of the final network for a 128 × 128 input (shapes and parameter counts read from the PyTorch model).

Layer	Operation	Output (C × H × W)	Params
input	BGR photo resized to 128 x 128 (cv2.INTER_AREA)	3 × 128 × 128	–
stem	3x3 conv, stride 2, BN, ReLU	48 × 64 × 64	1,392
enc1.0	DS block (stride 2)	64 × 32 × 32	3,728
enc1.1	DS block (stride 1, residual)	64 × 32 × 32	4,928
enc2.0	DS block (stride 2)	128 × 16 × 16	9,152
enc2.1	DS block (stride 1, residual)	128 × 16 × 16	18,048
enc2.2	DS block (stride 1, residual)	128 × 16 × 16	18,048
enc3.0	DS block (stride 2)	192 × 8 × 8	26,368
enc3.1	DS block (stride 1, residual)	192 × 8 × 8	39,360
enc3.2	DS block (stride 1, residual)	192 × 8 × 8	39,360
enc3.3	DS block (stride 1, residual)	192 × 8 × 8	39,360
context	global avg pool, 1x1 conv, add to every position	192 × 8 × 8	37,056
dec2	bilinear x2, concat enc2 (192+128 ch), DS block	128 × 16 × 16	44,736
dec1	bilinear x2, concat enc1 (128+64 ch), DS block	64 × 32 × 32	14,528
dec0	bilinear x2, concat stem (64+48 ch), DS block	48 × 64 × 64	6,704
head	1x1 conv to one object logit	1 × 64 × 64	49
output	bilinear x4 (cv2.resize), threshold logit > 0	1 × 256 × 256	–
	Total (with BatchNorm / after folding BN into convs)		**302,817 / 299,489**

The head predicts one object logit per pixel of a 64 × 64 grid. It is upsampled bilinearly to 256 × 256 and thresholded at 0 (probability 0.5). With BatchNorm folded into the convolutions (Chapter 6) the deployed network has 299,489 parameters.

Chapter 4

Training

4.1 Loss

Let p_i be the predicted object probability of pixel i (after upsampling to 256 × 256) and y_i (0 or 1) the true label. The loss is a weighted pixel-wise binary cross entropy plus soft Dice:

$$L = (1/N)\,\Sigma_i\, w_i\,[\,-y_i \log p_i - (1 - y_i)\log(1 - p_i)\,] \;+\; 1 - (2\,\Sigma_i\, p_i\, y_i + 1) / (\Sigma_i\, p_i + \Sigma_i\, y_i + 1)$$

$$w_i = 1 + 2 \cdot [\text{ pixel i lies within 2 px of the true outline }]$$

A single logit with binary cross entropy is the two-class pixel-wise cross entropy written with one output channel. Soft Dice (computed per image) balances small and large objects. The boundary weight w_i triples the cost of mistakes near the outline, because Boundary F1 only rewards outlines within a 5 × 5 window; the near-outline region is found with the same boundary map and 5 × 5 max-pool used by the metric.

4.2 Optimisation and Augmentation

Table 4.1: Training settings.

Setting	Value
Optimiser	AdamW, learning rate 3×10^{-3}, weight decay 10^{-4}
Schedule	1 warm-up epoch, then cosine decay to 0
Batch size / epochs	32 / 60 (about 25 s per epoch, 25 min in total)
Precision	mixed float16 on the GPU
Input to the network	256 × 256 photo area-downscaled to 128 × 128
Model selection	epoch with the best validation Q (epoch 59)

All augmentation runs on the GPU on whole batches, with the same geometric warp for photo and mask:

- **Geometric:** random zoom 0.8–1.45, aspect ratio ±15%, rotation ±12°, shift ±12%, horizontal flip.
- **Photometric:** brightness, contrast and saturation ±35%, a per-channel colour cast (to mimic tube light versus daylight), gamma 0.7–1.4, occasional blur and sensor noise.

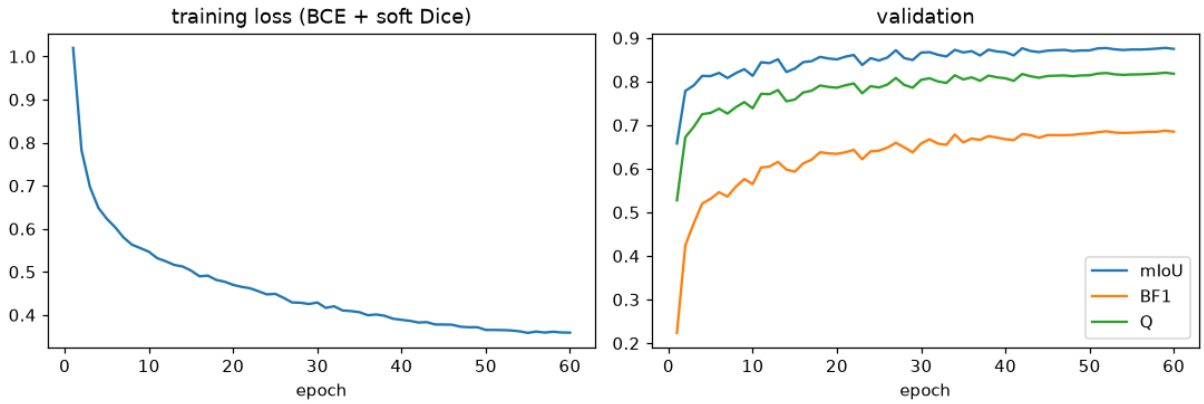


Figure 4.1: Training loss and validation metrics per epoch.

Chapter 5

Evaluation Metrics

5.1 Mean Intersection over Union

One 2 × 2 confusion matrix (TP, FP, FN, TN) is summed over all images of a set. Then

$$\mathrm{IoU}_{obj} = TP / (TP + FP + FN), \quad \mathrm{IoU}_{bg} = TN / (TN + FN + FP), \quad mIoU = (\mathrm{IoU}_{obj} + \mathrm{IoU}_{bg}) / 2.$$

5.2 Boundary F1

A boundary pixel is an object pixel with a background pixel directly above, below, left or right (pixels outside the image count as background). It is found by comparing the zero-padded mask with its four shifted copies. A predicted boundary pixel is correct if a true boundary pixel lies in its 5 × 5 window, and a true boundary pixel is found if a predicted one lies in its window:

$$P = (\text{correct predicted boundary pixels}) / (\text{predicted boundary pixels}), \quad R = (\text{found true boundary pixels}) / (\text{true boundary pixels}), \quad BF1 = 2PR / (P + R).$$

Looping over every pixel and its window in Python is far too slow, so the window test is vectorised: `F.max_pool2d(kernel 5, stride 1, padding 2)` applied to a boundary map gives, at every pixel, whether any boundary pixel lies in its 5 × 5 window. BF1 is computed per image on whole batches and averaged over the set. If neither mask has boundary pixels BF1 is 1, and if only one does it is 0.

Correctness check: `tests/test_metrics.py` compares both metrics with a direct pixel-by-pixel implementation of the definitions on 34 random and edge-case masks (empty, identical, full); all values match exactly.

Chapter 6

Keeping the Pipeline Fast

The timed part starts with a 256 × 256 photo in memory and ends with its 256 × 256 mask of 0s and 1s in memory (Figure 6.1). Every step in between counts, including the downscaling.

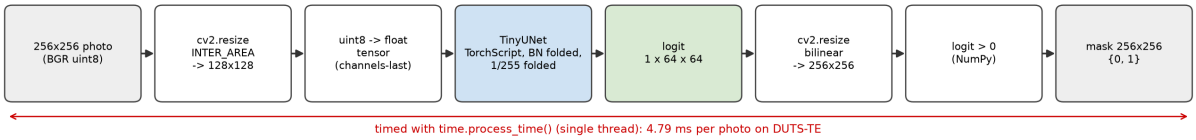


Figure 6.1: The timed CPU pipeline.

6.1 Speed Measures

- **BatchNorm folding.** Each BN after a convolution is folded into it: with BN scale γ, shift β, running mean μ and variance σ^2, the new weights are $W' = W \cdot \gamma / \sqrt{(\sigma^2 + \varepsilon)}$ and bias $b' = \beta - \mu\,\gamma / \sqrt{(\sigma^2 + \varepsilon)}$. The 1/255 input scaling is folded into the stem weights too, so raw pixel values go straight into the network.
- **Low working resolution.** The network sees 128 × 128 and predicts 64 × 64, about 4× less work than 256 × 256. The validation sweep (Chapter 7) showed that a 256 input does not pay for its cost.
- **Channels-last memory layout.** oneDNN runs small depthwise and 1 × 1 convolutions much faster in this layout (for example 21.6 ms to 13.5 ms for one sweep model in an early, loaded test).
- **TorchScript trace and freeze** remove the Python overhead between layers. Three warm-up calls on a blank input are made while loading, because TorchScript re-optimises the graph during its first calls.
- **Upsampling outside PyTorch.** Profiling showed that `F.interpolate` of the one-channel logit to 256 × 256 plus the comparison took about 1.5 ms. `cv2.resize` (bilinear, same half-pixel convention) and a NumPy threshold take about 0.1 ms.

6.2 Parameter File

Convolution weights are stored as int8 with one float32 scale per output channel c, and biases as float16:

$$s_c = \max |W_c| / 127, \quad q = \mathrm{round}(W_c / s_c), \text{ an integer from } -127 \text{ to } 127, \quad W_c \approx q \cdot s_c.$$

The weights are converted back to float32 once at load time (not timed), so inference itself is ordinary float32. On the validation set this gives the same Q as float16 storage (0.8128 versus 0.8127) while the file shrinks from 615 KB to 341 KB. The final file is **336,642 bytes**.

6.3 Timing

T_{CPU} is the sum of `time.process_time()` over the 5,019 DUTS-TE photos, one at a time after one untimed warm-up run, divided by 5,019: **4.79 ms** per photo. Two practical notes: (1) on Windows `time.process_time()` advances in 15.6 ms ticks, so the 20-run average printed by `predict.py` is coarse; the 5,019-photo average is the reliable figure. (2) The laptop CPU is about twice as slow on battery power: a repeat of the timing on battery gave 9.84 ms per photo. Both are far below the 50 ms limit.

Chapter 7

Design Choices on the Validation Set

Each candidate was trained for 50 epochs with the same split. T is the deployed pipeline timed on 500 validation photos with the CPU otherwise idle, and valCES uses validation Q in place of DUTS-TE Q. DUTS-TE played no part in these choices.

Table 7.1: Architecture and loss sweep (validation set). “deep” uses 2-4-6 blocks per encoder stage instead of 2-3-4.

Run	Input	Widths	Params	Loss	T (ms)	mIoU	BF1	Q	valCES
r128_w16	128	16-24-40-64	30K	BCE+Dice	1.84	0.840	0.564	0.757	0.750
r256_w16	256	16-24-40-64	30K	BCE+Dice	4.00	0.837	0.584	0.761	0.748
r192_w24	192	24-32-64-96	65K	BCE+Dice	3.84	0.860	0.628	0.790	0.776
r128_w32	128	32-48-96-128	150K	BCE+Dice	3.06	0.865	0.640	0.797	0.782
r128_w32 deep	128	32-48-96-160	260K	BCE+Dice	3.53	0.869	0.650	0.803	0.781
r128_w32	128	32-48-96-128	150K	+boundary	3.00	0.868	0.656	0.805	0.789
r128_w48	128	48-64-128-192	299K	BCE+Dice	4.09	0.874	0.670	0.813	0.787
final	**128**	**48-64-128-192**	**299K**	**+boundary**	–	**0.877**	**0.687**	**0.820**	–

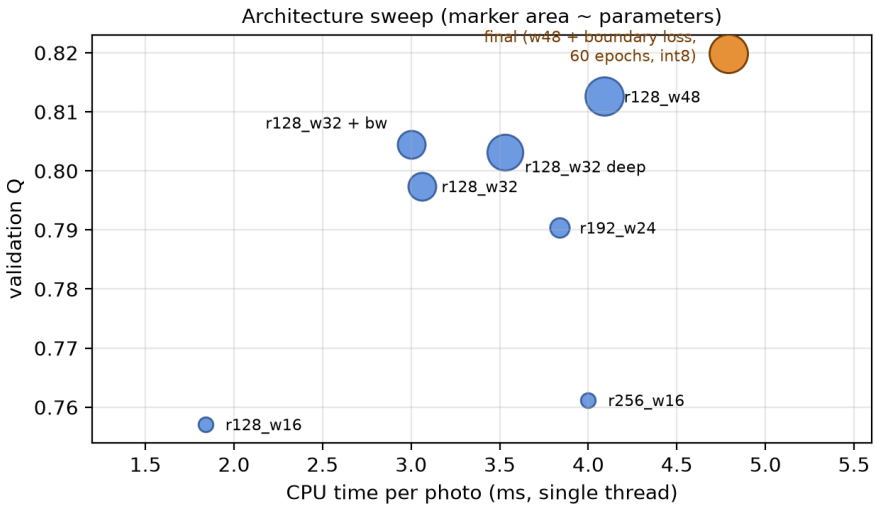


Figure 7.1: Validation Q against CPU time; marker area grows with the number of parameters.

Findings. (1) Width matters much more than input resolution: running at 256 instead of 128 doubles the time for +0.004 Q, while doubling the width gains +0.04 Q. (2) More depth helps less than more width. (3) The boundary weight adds about +0.007 Q, almost all of it through BF1. (4) At these sizes time and size cost only 1–3% of Q, so the final model is the widest one tested, with the boundary loss, trained for 60 epochs and stored as int8.

Chapter 8

Results

Table 8.1: Segmentation quality.

Set	Images	mIoU	Boundary F1	Q
Validation (held out from DUTS-TR)	1,053	0.8767	0.6873	0.8199
DUTS-TE	5,019	**0.7249**	**0.5069**	**0.6595**
Captured photos	4	0.8156	0.4165	0.6959

Table 8.2: Cost and final score.

T_{CPU} (DUTS-TE)	S_{disk}	Q (DUTS-TE)	CES
4.79 ms	336,642 B (0.32 MiB)	0.6595	**0.6416**

$$\mathrm{CES} = 0.6595 \times (1 - 0.20 \cdot 4.79/50 - 0.10 \cdot 336{,}642/4{,}194{,}304) = 0.6595 \times 0.9728 = \mathbf{0.6416}.$$

8.1 Validation versus DUTS-TE

Q falls from 0.82 on validation to 0.66 on DUTS-TE. The cached DUTS-TE masks were checked against the raw files and are aligned with their photos, so the gap is a real change of distribution:

- Objects cover on average **27%** of a DUTS-TR photo but only **15%** of a DUTS-TE photo.
- **18.5%** of DUTS-TE photos get object IoU below 0.3, against 3% on validation.
- DUTS-TE comes from different image sources, with more clutter, small objects and several objects.

Validation comes from the same distribution as training, so it overestimates performance on new data; this is why the assignment asks for both numbers.

Chapter 9

Captured Photos

Each photo was resized with the rule of Section 2.1 and its mask drawn on the resized photo with a small polygon tool (`src/annotate.py`). The photos are used for testing only. In Figure 9.1 the true mask is green and the prediction magenta.

Table 9.1: Captured photos.

Photo	Conditions	Object IoU	BF1
bottle	indoor, tube light, near, cluttered desk	0.92	0.70
cat	outdoor, harsh daylight, grass and shadows	0.12	0.19
chair	indoor, near, cluttered background	0.86	0.39
shoes	indoor, warm light, very near, cut by the frame	0.70	0.38



Figure 9.1: Captured photos: true mask (green) and predicted mask (magenta) as semi-transparent overlays.

Chapter 10

Failure Cases

10.1 Camouflage in a Cluttered Outdoor Scene

In the **cat** photo (IoU 0.12) the network marks the pink flowers along the wall and only the cat's face. The grey-brown fur has the same colours and texture as the dry grass and soil, under harsh daylight with hard shadows, while the flowers are the most colourful, high-contrast region. A network of about 300K parameters trained from scratch on 9,500 photos learns mostly colour and contrast cues and little about object categories, so it cannot tell that the cat matters more.

10.2 Small Object on a Large Textured Surface

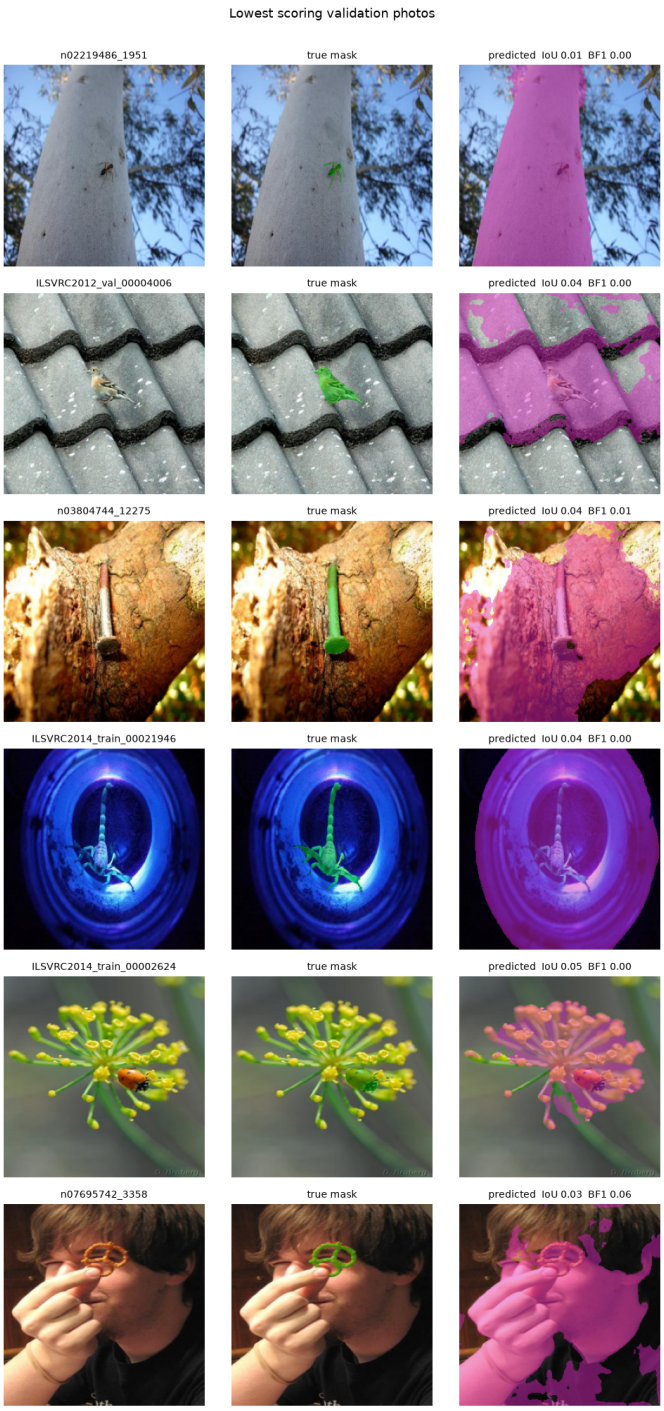


Figure 10.1: The six lowest-scoring validation photos.

In all six worst validation photos the true object is **small** (an ant on a tree trunk, a bird on a roof, a nail in bark, a ladybird on a flower, a pretzel in front of a face) and the network segments the **large surface** instead. Its strongest cue is a big, central region that differs from the border, and at a 64 × 64 output a 10-pixel object shrinks to two or three cells. The same cause explains most of the DUTS-TE gap, since DUTS-TE contains many more small objects.

10.3 Object Touching the Border under a Colour Cast

In the **shoes** photo (IoU 0.70, BF1 0.38) the shoes fill the frame and are cut by its left and bottom edges. The network misses the dark heel and the part that touches the border. Most DUTS objects lie away from the border, so the network has learned that border regions are usually background, and the dark leather in strong warm light is close in colour to the wooden floor.

10.4 Fine Structures and Boundary Accuracy

Even when the region is right, the outline can be off: the **chair** has IoU 0.86 but BF1 0.39. The gaps between the arm-rests and the seat and the cane mesh are thinner than one cell of the 64 × 64 output grid, and hand-drawn polygon masks are themselves only accurate to a few pixels.

Chapter 11

Conclusion

A tiny U-Net with depthwise separable convolutions, global context and concatenated skip connections, trained from scratch on 9,500 DUTS photos, segments the main object with validation Q = 0.82 and DUTS-TE Q = 0.66. The CPU pipeline needs 4.79 ms per photo on one thread and a 337 KB parameter file, which gives **CES = 0.642**.

The most useful lessons were that width matters more than input resolution, that a cheap boundary weight in the loss improves Boundary F1, and that most of the CPU time was overhead (upsampling, memory layout, Python) rather than convolutions, so profiling paid off more than shrinking the network. The main weakness is small objects and objects that look like their background, which is also why DUTS-TE scores lower than validation.

Table 11.1: Main files of the submission.

File	Purpose
`predict.py`	segment one photo, print T CPU ms and S disk bytes
`weights/model.pt`	parameter file (int8 conv weights + scales)
`src/model.py`	TinyUNet and BatchNorm folding
`src/train.py`	training with GPU augmentation
`src/metrics.py`	mIoU, Boundary F1, CES
`src/pipeline.py`	the timed CPU pipeline
`src/time_cpu.py`	official T_{CPU} on DUTS-TE
`src/evaluate.py`	metrics for validation, DUTS-TE and captured photos